ChronoHive Evaluation Package

Evaluation quickstart

CONFIDENTIAL — PROPRIETARY
Licensed solely for evaluation under the ChronoHive Terms of Confidentiality and the ChronoHive Evaluation License.
Do not distribute, copy, or publicly host.

© 2026 Layer1Labs Silicon Inc. All rights reserved.

© 2026 Layer1Labs Silicon Inc. All rights reserved.

A note on what this package is. This repository is Layer1Labs' public evaluator package, published openly so that prospective evaluators can read the evaluation terms, the API documentation, and the worked example before requesting an Evaluation License. Access to the hosted evaluation API is governed by the Terms of Confidentiality (**TOC.md**) and the ChronoHive Evaluation License (**LICENSE**) in this repository. This is a client package: it contains no engine source code, and the specifics of any individual evaluation relationship are not part of this package.

Evaluate ChronoHive's storage-admission capability. This is a **client package**: Python clients and tools that drive the hosted ChronoHive eval API — there is no server to self-host in this repository. The authoritative eval API lives with the ChronoHive service; this package is how evaluators reach it.

The API serves **real admission decisions from the ChronoHive Runtime kernel** against a **simulated** storage backend, contention model, and storage API surface. Evaluation numbers are simulation outcomes, not hardware measurements.

Quickstart (~5 minutes)

```
# 1. Get an API key from your ChronoHive contact, then read the canonical TOC:
curl https://api.layer1labs.ai/v1/toc

# 2. Sign and execute the TOC (pure-Python ed25519, no dependencies):
python3 tools/sign_toc.py --api-url https://api.layer1labs.ai \
    --api-key "$CHRONOHIVE_API_KEY" --key-id <your-key-id> \
    --name "Jane Doe" --org "Example Corp" \
    --email jane@example.com --submit

# 3. Run the worked example end to end (TOC → scenario → live kernel
#    decisions → result fetch):
python3 examples/eval_walkthrough.py --api-url https://api.layer1labs.ai \
    --api-key "$CHRONOHIVE_API_KEY" --key-id <your-key-id> \
    --name "Jane Doe" --org "Example Corp" --email jane@example.com
```

The one rule: execute the TOC first

Every eval endpoint stays locked (**403 toc_acceptance_required**) until your API key has a verified Terms of Confidentiality acceptance on file. The flow:

1. Read the canonical TOC: `GET /v1/toc` (or `TOC.md` in this repo — same text, pinned by SHA-256).
2. Sign it with `tools/sign_toc.py` (generates an ed25519 keypair, binds the TOC hash to your key id and identity, writes `toc_signing.key` at mode 600 and `toc_acceptance.json`).
3. Submit the acceptance: `POST /v1/toc/accept`.
4. Keep `toc_signing.key` — it is your proof of execution. Never commit it anywhere.

What's in this package

Path	What it is
clients/compile_client.py	Complete-project LF→specification compile client: hosted API mode, local pinned-toolchain mode, and an offline --check self-test
examples/eval_walkthrough.py	Worked end-to-end admission example (TOC → scenario → live kernel decisions)
tools/sign_toc.py	TOC signing and submission tool (pure-Python ed25519)
tools/toc_common.py, tools/ed25519.py	Acceptance-token format and RFC 8032 ed25519 implementation
lf/	Complete LF workload project (IoCoordinator.lf), editor config, and a checked-in reference specification for offline verification
toolchain/	Pinned LF toolchain: vendored chronoc v0.1.0-alpha.1 binary + lfc v0.13.0 fetch (SHA-256 verified)
scripts/compile_lf.sh	LF→specification compile helper (self-contained; needs only a JVM for lfc)
scripts/demo-lf.sh	Guided LF tour: source → validate → compile → specification provenance
scripts/fetch-lfc.sh	One-time pinned-lfc download with hash verification

© 2026 Layer1Labs Silicon Inc. — CONFIDENTIAL — PROPRIETARY

docs/ARCHITECTURE.md	How it works: system architecture, request gating, and how to read your numbers (start here if you want the big picture)
docs/API.md	Full hosted API reference, including the /v1/lf/compile contract
docs/LF_TOOLCHAIN.md	Toolchain pins and the LF→specification pipeline
TOC.md	Terms of Confidentiality (pinned by SHA-256)
pdf/	Print-ready PDFs of every document: Quickstart, How It Works, API Reference, LF Toolchain, TOC

Compiling an LF workload to a specification

Lingua Franca workloads compile to the **.cspec** v1 artifact the ChronoHive engine executes. The compile takes the **complete LF project** — every **.lf** file as a file map — plus an entrypoint, parameters, and capacities:

```
python3 clients/compile_client.py --api-url https://api.layer1labs.ai \
    --api-key "$CHRONOHIVE_API_KEY" --capacity storage_bw=100 -o /tmp/io.cspec
```

The hosted compile API requires a key with a **compile license** (issued by the operator) on top of TOC execution. It enforces the same contract the client does: ≤64 files, ≤512 KiB per file, ≤2 MiB aggregate, relative **.lf** paths only, 120 s compile timeout.

For a guided tour of the same chain (source → pinned-**lfc** validation → **chronoc** lowering → specification provenance), run:

```
scripts/demo-lf.sh
```

or compile the checked-in project directly with the vendored toolchain (self-contained — needs only a JVM for **lfc**):

© 2026 Layer1Labs Silicon Inc. — CONFIDENTIAL — PROPRIETARY

```
scripts/compile_lf.sh lf/IoCoordinator.lf -o io.cspec --capacity storage_bw=100
```

Full authoring setup (IDE extension, Java) is documented in **docs/LF_TOOLCHAIN.md**. Evaluation itself needs neither — the API, the worked example, and the scenarios all run on stdlib Python.

Compiling is optional for evaluation — the admission API needs neither **lfc** nor **chronoc**. The local mode (**local**) and the offline check (**check**, which verifies the checked-in reference specification byte-structure with no network and no **chronoc** — pinned **lfc** + a JRE are still required) cover evaluators who work air-gapped.

Verifying the package

```
python3 scripts/check.py
```

The gate byte-compiles every Python file, reproduces the RFC 8032 ed25519 test vectors, runs the branding sweep (vendor-neutrality, no internal markers), and runs the compile client's **check** (LF project map, pinned **lfc** validation gate, reference-specification verification).

License and confidentiality

This package is **proprietary and confidential**. Your rights are limited to evaluation under **TOC.md** and **LICENSE** — no production use, no redistribution, no public hosting. See **NOTICE** for the honesty notes that apply to every evaluation number.

© 2026 Layer1Labs Silicon Inc. — CONFIDENTIAL — PROPRIETARY